# Website Technical Brief

*Implementation requirements and open technical decisions for the new 12-page ITI website*

Purpose: provide a neutral technical brief for deciding how Google AI Studio should build the site. This document records the user's stated requirements and separates fixed requirements from choices that should remain open until the prompt is approved.

## 1. Project scope

- A college/institute website with approximately 12 pages.
- Begin implementation with the homepage only; designs for other pages will be provided later.
- Figma designs exist for desktop and mobile homepage layouts.
- The site includes static informational content, a Sanity-powered notice/announcement section, and multiple contact/inquiry forms.
- The site is not expected to receive substantial feature updates after delivery.
- The priority is to deliver quickly, keep the implementation simple, and preserve the ability to export the source code and debug it with Codex or another coding agent.

## 2. Functional requirements

### Notices

- Notices must be designed for future integration with Sanity CMS.
- Google AI Studio should build the UI and prepare a clear integration seam, but should not configure Sanity project credentials, datasets, schemas or production connections during the initial generation phase.
- Do not replace the intended CMS functionality with an unmarked permanent hardcoded solution. If mock notice data is needed for the UI, keep it clearly separated and easy to replace.
- Design for loading, empty and error states, even if the live integration is deferred.

### Contact forms

- All contact forms are intended to save submissions into Sanity and send an email to the client's email address using Resend.
- Google AI Studio should prepare the form UI, field structure, client-side validation and integration boundaries, but should not configure production secrets or connect the live Sanity/Resend services yet.
- Never put API keys or secret tokens in browser-side code.
- Provide clear pending, success and failure states. Do not show success unless the server-side submission succeeds.
- The eventual server-side handler should validate and sanitize submitted data before writing to Sanity or sending email.

### Static pages

- Other informational pages can use static content unless a later requirement says otherwise.
- Do not introduce a CMS-driven page builder for all pages unless there is a concrete need.

# 3. Framework and architecture decision principles

No framework has been approved yet. Google AI Studio may recommend a suitable stack, but the decision should optimize for a fast, lightweight production site, straightforward export/debugging, local assets, and deferred Sanity/Resend integration.

| Criterion | Requirement / preference |
|---|---|
| **Runtime performance** | Very fast, especially on mobile; target Lighthouse Performance above 90 where practical, without guaranteeing a score before testing. |
| **Complexity** | Prefer the simplest implementation that meets the requirements; no unnecessary state management or elaborate abstraction. |
| **Exportability** | The generated source must be a normal, editable codebase that can be exported and debugged with Codex, Claude or other tools. |
| **Styling** | Use Tailwind CSS; centralize design tokens so brand adjustments do not require editing every page. |
| **CMS** | Prepare for Sanity, but defer actual project configuration and credentials. |
| **Email** | Prepare for server-side Resend integration, but defer credentials and live setup. |
| **Assets** | Local images and local icon assets only; no remote image or icon fetching. |
| **Responsive design** | Use both supplied Figma desktop and mobile screens as requirements. |
| **Scope control** | Implement the homepage first. Do not invent the remaining page designs. |

# 4. Performance requirements

- Prefer server-rendered or statically generated output where suitable; avoid shipping a large client-side application just to render mostly static pages.
- Use client-side JavaScript only for interactions that need it.
- Avoid unnecessary UI libraries, animation packages, icon packages and dependencies.
- Use local images, explicit dimensions/aspect ratios, and responsive image sizes where appropriate.
- Load fonts locally and use font-display: swap; avoid blocking external font requests.
- Lazy-load below-the-fold imagery where appropriate, but do not lazy-load the main above-the-fold hero image if doing so delays the Largest Contentful Paint.
- Keep the hero and initial viewport lightweight; avoid large autoplay video backgrounds and heavy carousels unless the design truly requires them.
- Validate the production build and measure Lighthouse on the deployed production-like site. A score above 90 is a target, not a promise.

# 5. Asset requirements

### Images

- Use one local placeholder image in all image slots during the initial implementation.
- Do not source images from web URLs or remote stock-image services.
- Use a single clearly named local placeholder asset and make it straightforward for the user to replace individual image references manually later.
- Do not embed remote image URLs in data objects or components.

### Icons

- Do not fetch icons from the web or remote icon APIs.
- Prefer local SVG files or a simple central icon-reference approach so the user can replace icons with their Figma exports later.

- Do not draw complex icons using emoji or text glyphs as a substitute for the intended icon design.

### Fonts and language

- The site uses both Hindi and English.
- The user will update the final text and add the appropriate Hindi font later.
- Use Tailwind font utilities or theme tokens to control font-family, so font changes are centralized rather than hardcoded in each component.
- The supplied screenshot lists Hind and Montserrat local WOFF2 files at weights 300, 400 (regular), 500 and 600.
- The listed files are named with a latin subset. Verify script coverage before assuming they can render Hindi/Devanagari characters; do not silently claim the supplied files contain Devanagari glyphs.
- Copy the text that can be read from the screenshots where practical, but do not spend time guessing unclear text. Use clear placeholders for illegible copy and keep it easy to edit.

## 6. Tailwind and design-system requirements

- Tailwind CSS is required, but the exact Tailwind major version and configuration format have not yet been approved.
- Use a single, documented source of truth for colours, font families, typography scale, spacing, breakpoints, radii and shadows.
- If Tailwind v4 is selected, use its CSS-first @theme configuration where appropriate; do not create a redundant v3-style configuration unless required.
- If Tailwind v3 is selected, use its supported configuration file and extend the theme there.
- Components should consume named theme tokens rather than repeating arbitrary hex values and font declarations.
- Use Figma as the visual authority. Screenshot-based colours and measurements are estimates unless confirmed from Figma values.
- Keep architecture light: reusable header/footer/cards/forms are reasonable, but don't force every section into a generic abstraction.

## 7. What Google AI Studio should not do in the first phase

- Do not configure a real Sanity project, dataset, schema deployment, credentials or production connection.
- Do not configure Resend credentials or send real email during the initial build.
- Do not invent or expose API keys, environment-variable values, client secrets or tokens.
- Do not fetch images or icons from remote URLs.
- Do not implement all 12 pages based on assumptions; build the homepage only until further designs are supplied.
- Do not invent detailed page content where screenshots are illegible.
- Do not add large dependencies or complex abstractions without a clear requirement.
- Do not claim the Lighthouse score exceeds 90 unless it has actually been measured.
- Do not make the application depend on a Google AI Studio-only runtime; keep it exportable as a conventional project.

## 8. Suggested deliverables from the initial AI Studio build

- A complete homepage matching the desktop and mobile Figma references as closely as practical.
- A working responsive navigation and any clearly depicted UI interactions.

- A centralized Tailwind theme/design-token setup.
- Local placeholder image usage and replaceable local icon references.
- Editable static text and clearly separated mock data for notices.
- Contact forms with front-end validation and clearly marked server-side integration points, without live credentials.
- A project README describing install, run, build, export and later Sanity/Resend integration steps.
- A concise summary of the chosen framework, reasons for the choice, performance risks and how to replace images, icons, fonts and text.

## 9. Decisions still requiring user approval

- Framework and rendering approach.
- Exact Tailwind major version and token configuration mechanism.
- Whether notices should show mock data, a loading state, or a clearly labelled placeholder until Sanity is connected.
- Exact server-side form integration architecture for later Sanity writes and Resend email.
- Final brand colours and font-family assignments from Figma.
- Final project deployment target and environment setup.

## 10. Source basis and limitations

This brief is based on the user's stated requirements, the supplied homepage screenshots, the screenshot of local font filenames, and the supplied global.css from the previous project. The previous global.css uses Tailwind CSS v4's @theme syntax and defines an older burgundy/maroon/sage/cream palette; that palette is not assumed to be the new website's palette. Exact screenshot-derived colours are provisional. No live Sanity or Resend configuration has been inspected for the new project.